

Python for Data Science

Buổi 9: Trực Quan Hóa Dữ Liệu II & EDA Thực Hành

PGS-TS Nguyễn Mạnh Hùng

August 19, 2026

Cấu trúc Buổi 9

- 1 Ôn tập Buổi 8 và đặt vấn đề
- 2 Module 40: Nhập môn Seaborn Module 41: Phân phối thống kê
- 3 Module 42: Pair Plots Module 43: Faceted Histograms
- 4 Module 44: Categorical & Bar Plots Module 45: Joint Distributions
- 5 Module 46–47: Ứng dụng thật – bit.ly/USA.gov (múi giờ & dữ liệu địa lý)
- 6 Module 48–49: Ứng dụng thật EDA – Marathon Finishing Times
- 7 Module 50–51: Trực quan hóa 3 chiều (3D Plotting)
- 8 Module 52–53: EDA case study lớn – US Baby Names 1880–2010
- 9 Bài tập áp dụng, liên hệ Đồ án giữa kỳ

Nhắc lại Buổi 8

- Buổi 8 đã trang bị Matplotlib nền tảng: line/scatter, error bar, contour, histogram, legend, colorbar, subplots, text/annotation, tick & style.
- Toàn bộ đó là **giao diện cấp thấp** – kiểm soát chi tiết từng điểm ảnh, nhưng cần nhiều dòng code cho các biểu đồ *thống kê* thông dụng (phân phối, tương quan nhiều biến, so sánh nhóm).

Vấn đề

Muốn vẽ “phân phối của biến X theo từng nhóm giới tính, kèm khoảng tin cậy hồi quy” bằng Matplotlib thuần cần hàng chục dòng code. Ta cần một giao diện cấp cao hơn, hiểu trực tiếp DataFrame của Pandas.

Buổi 9 giới thiệu **Seaborn** (xây trên Matplotlib), sau đó mở rộng sang **trực quan hóa 3D**, dữ liệu **địa lý**, và tổng hợp mọi kỹ năng Buổi 1–9 vào một quy trình **EDA end-to-end** trên các case study thực tế – chuẩn bị trực tiếp cho **Đồ án giữa kỳ**.

Lộ trình dữ liệu thật của Buổi 9

Case study	Nguồn dữ liệu thật	Kỹ thuật minh họa
1.usa.gov (bit.ly)	datasets/bitly_usagov (PDA)	Seaborn barplot, dữ liệu địa lý
Marathon Finishing Times	jakevdp/marathon-data (GitHub)	jointplot, PairGrid, violin, Implot
US Baby Names 1880–2010	datasets/babynames (PDA)	Toàn bộ quy trình EDA end-to-end
Iris, Tips, Planets	Bộ dữ liệu tích hợp Seaborn	pairplot, FacetGrid, catplot, joint

Toàn bộ dữ liệu và code trong buổi này đều chạy được thật – không có số liệu minh họa giả định.

Module 40: Nhập môn Seaborn

Chuẩn đầu ra (Learning Outcomes)

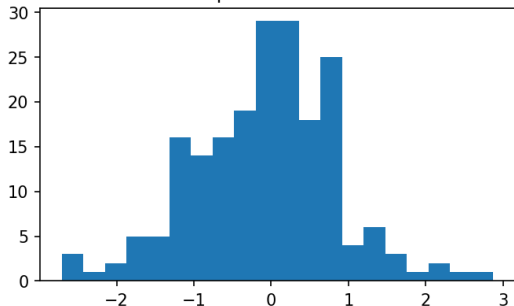
Sau module này, học viên có khả năng:

- 1 Giải thích quan hệ giữa Seaborn và Matplotlib.
 - 2 Áp dụng `sns.set_theme()` để đổi giao diện mặc định.
 - 3 Nhận biết khi nào nên dùng Seaborn thay vì Matplotlib thuần.
- Seaborn **không thay thế** Matplotlib – nó cung cấp API cấp cao, thao tác trực tiếp trên `DataFrame`, tự động suy luận trục/màu/legend từ tên cột, và tích hợp sẵn các phép ước lượng thống kê (KDE, hồi quy, khoảng tin cậy).
 - Mọi đối tượng Seaborn trả về vẫn là `Figure/Axes` của Matplotlib – có thể tùy chỉnh tiếp bằng các lệnh đã học ở Buổi 8.

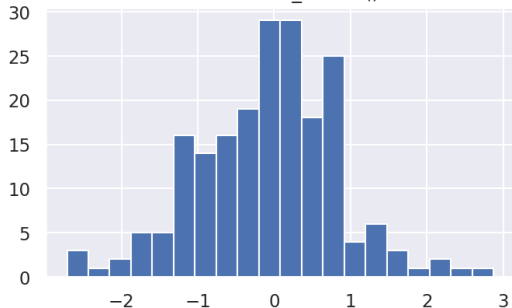
Hiệu ứng sns.set_theme()

```
import seaborn as sns
# Trước: biểu đồ Matplotlib mặc định
ax.hist(x, bins=20)
# Sau: gọi 1 lần duy nhất, ảnh hưởng toàn bộ phiên làm việc
sns.set_theme()
ax.hist(x, bins=20)          # cùng 1 lệnh, giao diện khác nhau
```

Matplotlib mặc định



Sau sns.set_theme()



Bốn họ hàm chính của Seaborn

Họ hàm	Ví dụ	Mục đích
Phân phối (distribution)	histplot, kdeplot, displot	Phân phối 1–2 biến
Quan hệ (relational)	scatterplot, lineplot	Quan hệ giữa 2 biến số
Phân loại (categorical)	catplot, boxplot, violinplot	So sánh theo nhóm rời rạc
Đa biến (multi-plot grid)	pairplot, jointplot, FacetGrid	Lưới nhiều biểu đồ con

Buổi 9 lần lượt đi qua cả 4 họ hàm này, minh họa trên dữ liệu thật.

Tổng kết Module 40

- Seaborn = API thống kê cấp cao, xây trên Matplotlib, làm việc trực tiếp với DataFrame.
- `sns.set_theme()`: đổi giao diện mặc định toàn phiên làm việc, chỉ cần gọi 1 lần.
- 4 họ hàm: phân phối, quan hệ, phân loại, lưới đa biểu đồ – mỗi họ giải quyết 1 nhu cầu EDA khác nhau.
- Kết quả trả về vẫn tương thích Matplotlib – có thể phối hợp với Buổi 8.

Module 41: Phân phối thống kê với Seaborn

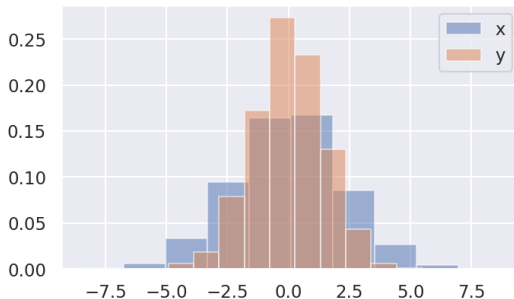
Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Vẽ histogram nhiều nhóm chồng bằng `alpha`.
- 2 Ước lượng mật độ (KDE) 1 chiều và 2 chiều bằng `sns.kdeplot()`.
- 3 Giải thích ý nghĩa của `fill=True` và tham số `shade` đã lỗi thời.

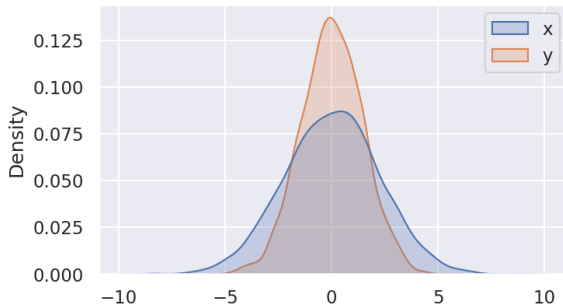
Histogram nhiều nhóm chồng lên nhau

```
data = rng.multivariate_normal([0, 0], [[5, 2], [2, 2]], size=2000)
data = pd.DataFrame(data, columns=['x', 'y'])
for col in 'xy':
    ax.hist(data[col], density=True, alpha=0.5, label=col)
```



Kernel Density Estimation (KDE) 1 chiều

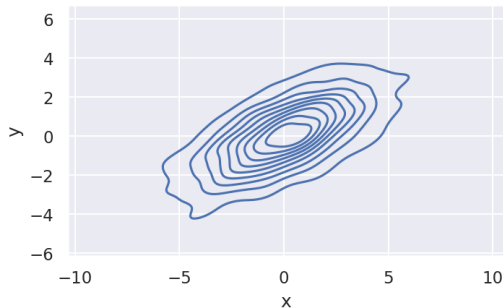
```
sns.kdeplot(data=data, fill=True)      # fill= thay the shade= da loi thoi
```



KDE cho một ước lượng *mượt* của hàm mật độ, thay vì histogram rời rạc phụ thuộc số lượng bin.

Mật độ đồng thời 2 chiều (Joint Density)

```
sns.kdeplot(data=data, x='x', y='y')    # truyền cả 2 trục -> mật độ 2D
```



Đường đồng mức biểu diễn vùng có mật độ điểm dữ liệu cao – tương tự `contour()` (Buổi 8) nhưng ước lượng trực tiếp từ mẫu, không cần hàm $f(x,y)$ tường minh.

Tổng kết Module 41

- `ax.hist(..., alpha=)`: so sánh nhiều phân phối trên cùng 1 trục.
- `sns.kdeplot(data, fill=True)`: ước lượng mật độ mượt 1 chiều.
- `sns.kdeplot(data, x=, y=)`: mật độ đồng thời 2 chiều, dạng đường đồng mức.
- Cả hai đều thao tác trực tiếp trên `DataFrame`, không cần tính toán trung gian như Matplotlib thuần.

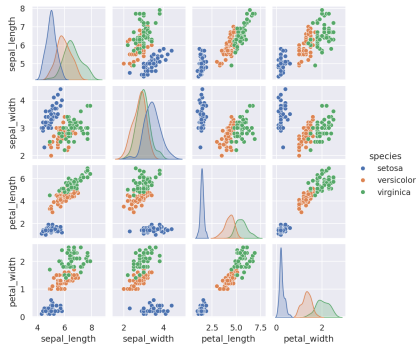
Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Sử dụng `sns.pairplot()` để khảo sát tương quan nhiều biến cùng lúc.
- 2 Đọc hiểu bố cục lưới: đường chéo = phân phối 1 biến, ô ngoài = tương quan cặp biến.

pairplot() trên bộ dữ liệu Iris

```
iris = sns.load_dataset("iris")  
sns.pairplot(iris, hue="species", height=2.5);
```



Đọc hiểu Pair Plot

- **Đường chéo:** phân phối (KDE) của từng biến, tách theo màu loài (`hue="species"`).
- **Ô ngoài đường chéo:** scatter plot giữa mỗi cặp biến – nhận ra ngay *setosa* (xanh) tách biệt hoàn toàn khỏi 2 loài còn lại theo chiều dài/rộng cánh hoa.
- Đây là bước **EDA đầu tiên** kinh điển khi nhận một bộ dữ liệu nhiều biến số mới – sẽ dùng lại thường xuyên ở Buổi 10–12 (Machine Learning).

- `sns.pairplot(df, hue=)`: ma trận scatter cho mọi cặp biến số, 1 dòng code.
- Thay thế nhanh cho việc gọi `ax.scatter()` thủ công $\binom{n}{2}$ lần như ở Buổi 8.
- Công cụ EDA đầu tay khi khám phá bộ dữ liệu nhiều chiều chưa quen thuộc.

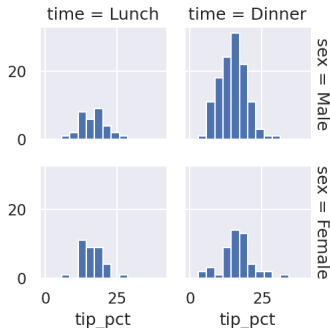
Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Dùng `sns.FacetGrid` để chia nhỏ dữ liệu thành lưới theo 1–2 biến phân loại.
- 2 Áp một hàm vẽ (Matplotlib hoặc Seaborn) lên từng ô lưới bằng `grid.map()`.

FacetGrid trên dữ liệu tiền boia nhà hàng (tips)

```
tips = sns.load_dataset('tips')
tips['tip_pct'] = 100 * tips['tip'] / tips['total_bill']
grid = sns.FacetGrid(tips, row="sex", col="time", margin_titles=True)
grid.map(plt.hist, "tip_pct", bins=np.linspace(0, 40, 15));
```



Đọc hiểu Faceted Histogram

- Lưới 2×2 : hàng = giới tính người phục vụ, cột = buổi (trưa/tối).
- Nhận ra ngay: dữ liệu buổi tối của nhân viên nam **nhều hơn hẳn** các ô khác – một quan sát về *cân bằng mẫu* quan trọng trước khi phân tích thống kê.
- Tỷ lệ tiền boa (`tip_pct`) tập trung quanh 10–20% ở mọi nhóm, có vài giá trị ngoại lai (outlier) cả hai đầu.

Tổng kết Module 43

- `sns.FacetGrid(df, row=, col=)`: tạo lưới Axes theo tổ hợp biến phân loại.
- `grid.map(ham_ve, "cot")`: áp 1 hàm vẽ (kể cả hàm Matplotlib thuần) lên từng ô.
- Faceting giúp phát hiện mất cân bằng dữ liệu giữa các nhóm con – bước kiểm tra quan trọng trước khi so sánh nhóm.

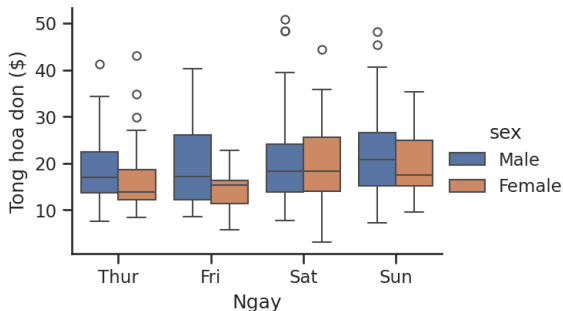
Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Vẽ box plot theo nhóm bằng `sns.catplot(..., kind="box")`.
- 2 Đếm số lượng bản ghi theo danh mục bằng `kind="count"`.
- 3 Thêm chiều thứ 3 vào biểu đồ danh mục bằng `hue=`.

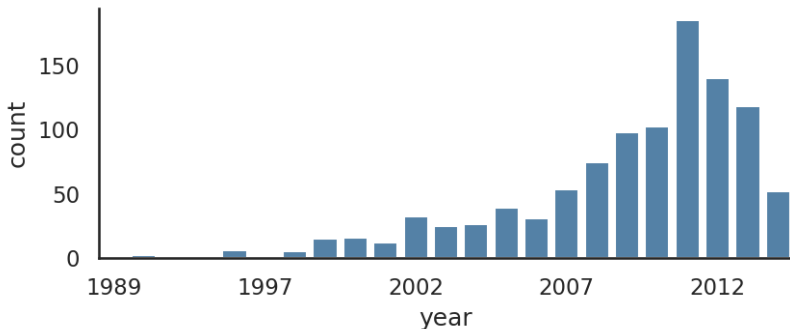
Box plot: tổng hóa đơn theo ngày và giới tính

```
with sns.axes_style(style='ticks'):  
    g = sns.catplot(x="day", y="total_bill", hue="sex", data=tips, kind="box")  
    g.set_axis_labels("Ngày", "Tổng hóa đơn")
```



Đếm số lượng: bộ dữ liệu Planets

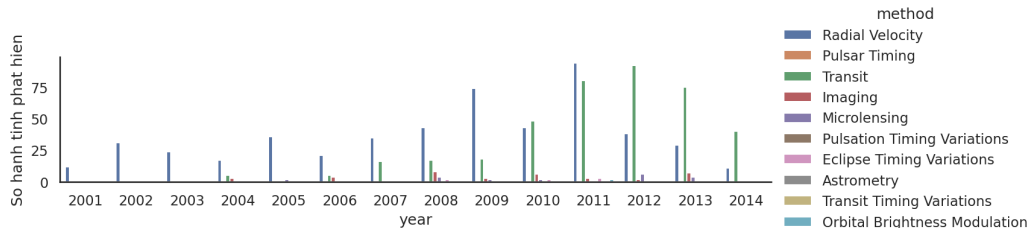
```
planets = sns.load_dataset('planets')  
with sns.axes_style('white'):  
    g = sns.catplot(x="year", data=planets, kind="count", color='steelblue')  
    g.set_xticklabels(step=5)
```



Số hành tinh ngoài hệ Mặt Trời được phát hiện tăng vọt sau 2009 – năm phóng vệ tinh Kepler.

Thêm chiều thứ 3: phương pháp phát hiện

```
with sns.axes_style('white'):
    g = sns.catplot(x="year", data=planets, kind='count',
                    hue='method', order=range(2001, 2015))
```



hue= tách mỗi cột theo phương pháp phát hiện – thấy rõ *Transit* (đo độ mờ sáng) trở thành phương pháp chủ đạo từ 2009.

Tổng kết Module 44

- `sns.catplot(..., kind="box"/"count"/...)`: 1 hàm, nhiều kiểu biểu đồ danh mục.
- `hue=`: thêm 1 biến phân loại thứ 3 vào cùng biểu đồ 2D.
- `sns.axes_style()`: context manager đổi style tạm thời – tương tự `plt.style.context()` (Buổi 8).

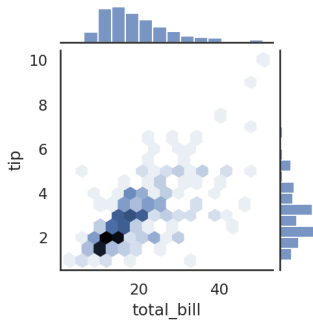
Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Vẽ đồng thời phân phối 2 biến và 2 phân phối biên bằng `sns.jointplot()`.
- 2 So sánh `kind="hex"` (mật độ) và `kind="reg"` (hồi quy tuyến tính tự động).

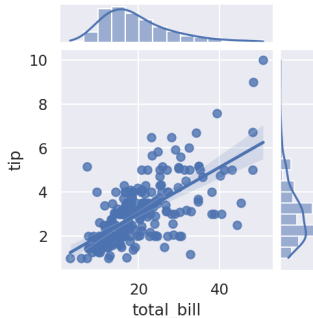
jointplot(kind="hex"): mật độ dạng lục giác

```
with sns.axes_style('white'):  
    sns.jointplot(x="total_bill", y="tip", data=tips, kind='hex')
```



jointplot(kind="reg"): tự động hồi quy

```
sns.jointplot(x="total_bill", y="tip", data=tips, kind='reg');
```



`kind="reg"` tự fit và vẽ đường hồi quy tuyến tính kèm khoảng tin cậy – xem trước kỹ thuật sẽ học sâu ở Buổi 11 (Supervised Learning).

- `sns.jointplot()`: kết hợp biểu đồ chính (giữa) và 2 phân phối biên (trên/phải) trong 1 lệnh.
- `kind="hex"`: phù hợp dữ liệu lớn, tránh chồng lấp điểm (so với scatter thường).
- `kind="reg"`: thêm ước lượng hồi quy tuyến tính tự động – hữu ích cho khám phá quan hệ tuyến tính sơ bộ.

Module 46: Ứng dụng thật – bit.ly/USA.gov (múi giờ)

Chuẩn đầu ra (Learning Outcomes)

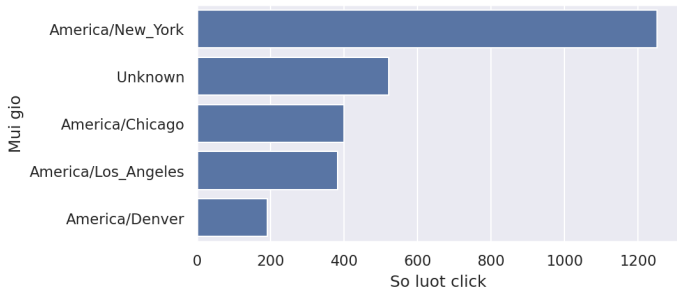
Sau module này, học viên có khả năng:

- 1 Đọc dữ liệu JSON dòng-theo-dòng (JSON Lines) thành DataFrame.
- 2 Kết hợp `value_counts()`, `groupby()` (Buổi 6) với `sns.barplot()` để trực quan hóa.

Dữ liệu thật: `datasets/bitly_usagov/example.txt` (PDA) – 3 560 bản ghi log click rút gọn liên kết `1.usa.gov`, thu thập năm 2012.

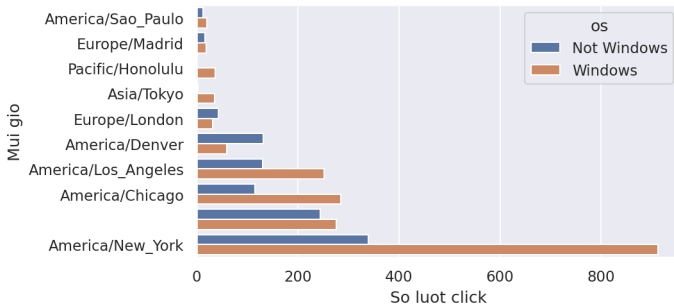
Nạp dữ liệu và đếm múi giờ phổ biến nhất

```
import json
records = [json.loads(line) for line in open(path)]
frame = pd.DataFrame(records)
clean_tz = frame["tz"].fillna("Missing")
clean_tz[clean_tz == ""] = "Unknown"
tz_counts = clean_tz.value_counts()
sns.barplot(y=tz_counts.head().index, x=tz_counts.head().to_numpy());
```



Thêm chiều: hệ điều hành theo múi giờ

```
cframe["os"] = np.where(cframe["a"].str.contains("Windows"),  
                        "Windows", "Not Windows")  
agg_counts = cframe.groupby(["tz", "os"]).size().unstack().fillna(0)  
count_subset = agg_counts.take(agg_counts.sum(axis="columns").argsort()[-10:])  
sns.barplot(x="total", y="tz", hue="os", data=count_subset.stack());
```



Tổng kết Module 46

- `json.loads()` theo từng dòng: mẫu chuẩn đọc dữ liệu JSON Lines (mỗi dòng 1 bản ghi độc lập).
- `value_counts() + sns.barplot()`: thay thế cho `ax.barh()` thủ công ở Buổi 8.
- `groupby(...).size().unstack()`: mẫu tổng hợp 2 chiều đã học ở Buổi 6, nay trực quan hóa trực tiếp bằng `hue=`.

Module 47: Dữ liệu địa lý (Geographic Data)

Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

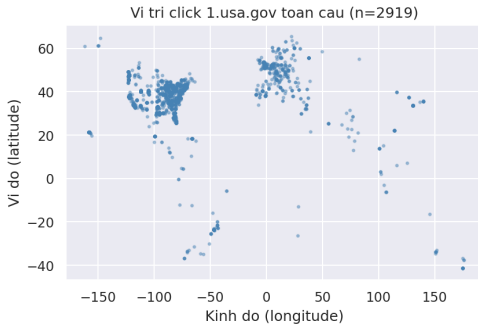
- 1 Vẽ dữ liệu có tọa độ kinh độ/vĩ độ bằng scatter plot đơn giản.
- 2 Hiểu vì sao cần thư viện chuyên dụng (Cartopy/GeoPandas) để có bản đồ nền đầy đủ.

Giới hạn công cụ

Handbook (04.13) minh họa bằng thư viện Basemap – nay đã ngừng phát triển. Các thư viện thay thế (Cartopy, GeoPandas) cần tải dữ liệu bản đồ nền (Natural Earth) qua mạng, không khả dụng ngoại tuyến trong môi trường này. Ta minh họa nguyên lý cốt lõi – **trục x = kinh độ, trục y = vĩ độ** – bằng scatter plot thuần Matplotlib trên dữ liệu thật.

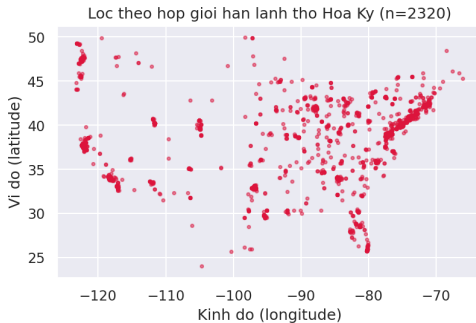
Tái sử dụng dữ liệu bit.ly: trường tọa độ 11

```
with_11 = frame[frame["11"].notna()]\nlats = with_11["11"].apply(lambda v: v[0])\nlons = with_11["11"].apply(lambda v: v[1])\nax.scatter(lons, lats, s=4, alpha=0.4)
```



Lọc theo hộp giới hạn lãnh thổ Hoa Kỳ

```
us_mask = (lats > 24) & (lats < 50) & (lons > -125) & (lons < -66)  
ax.scatter(lons[us_mask], lats[us_mask], s=6, alpha=0.5)
```



Chỉ với 2320 điểm click, hình dạng lãnh thổ Hoa Kỳ hiện rõ – minh chứng cho sức mạnh của trực quan hóa: mật độ điểm tự nó vẽ nên đường biên giới, không cần bản đồ nền.

Tổng kết Module 47

- Nguyên lý cơ bản của mọi bản đồ điểm: trục x = kinh độ (longitude), trục y = vĩ độ (latitude).
- Boolean mask (Buổi 2) lọc theo hộp giới hạn địa lý – kỹ thuật tổng quát, áp dụng được cho mọi vùng.
- Bản đồ nền thật (đường biên giới, địa hình) cần Cartopy/GeoPandas + dữ liệu Natural Earth – ngoài phạm vi ngoại tuyến của khóa học.

Module 48: Ứng dụng thật EDA – Marathon Finishing Times (I)

Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Chuyển đổi chuỗi thời gian dạng "hh:mm:ss" sang `timedelta` bằng `converters=`.
- 2 Tạo biến phái sinh (`split_frac`) phục vụ phân tích.

Dữ liệu thật: `jakevdp/marathon-data` (GitHub) – 37 250 người hoàn thành 1 giải marathon, gồm thời gian tại điểm nửa chặng (`split`) và về đích (`final`).

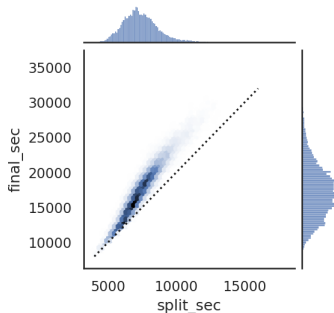
Nạp và chuyển đổi kiểu thời gian

```
def convert_time(s):  
    h, m, sec = map(int, s.split(':'))  
    return datetime.timedelta(hours=h, minutes=m, seconds=sec)  
  
data = pd.read_csv('marathon-data.csv',  
                  converters={'split': convert_time, 'final': convert_time})  
data['split_sec'] = data['split'].dt.total_seconds()  
data['final_sec'] = data['final'].dt.total_seconds()
```

Pandas đọc cột thời gian dạng chuỗi (object) mặc định – `converters=` áp hàm chuyển đổi ngay khi đọc, tránh phải xử lý lại sau (nguyên tắc *làm sạch sớm* đã nhấn mạnh từ Buổi 4).

Jointplot: thời gian nửa chặng vs. về đích

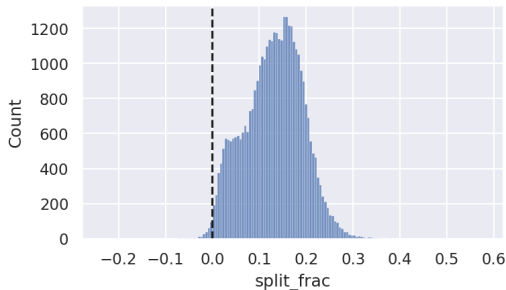
```
g = sns.jointplot(x='split_sec', y='final_sec', data=data, kind='hex')
g.ax_joint.plot(np.linspace(4000, 16000), np.linspace(8000, 32000), ':k')
```



Đường chấm là đường tham chiếu “giữ tốc độ đều”. Phần lớn điểm nằm *trên* đường này – đa số vận động viên **chậm lại** ở nửa sau.

Biến phái sinh: tỷ lệ chênh lệch tốc độ (split_frac)

```
data['split_frac'] = 1 - 2 * data['split_sec'] / data['final_sec']  
sns.histplot(data['split_frac'], kde=False)  
ax.axvline(0, color='k', linestyle='--')
```



$\text{split_frac} < 0$ nghĩa là chạy nửa sau *nhANH* hơn nửa đầu (“negative split”). Chỉ 251/37 250 người (0,67%) làm được điều này.

Tổng kết Module 48

- `pd.read_csv(..., converters=)`: chuyển đổi kiểu dữ liệu ngay khi đọc file.
- `Series.dt.total_seconds()`: trích giá trị số từ `timedelta` để tính toán/vẽ biểu đồ.
- Biến phái sinh (`split_frac`) biến câu hỏi định tính (“có chậm lại không?”) thành biến định lượng có thể trực quan hóa.

Module 49: Ứng dụng thật EDA – Marathon Finishing Times (II)

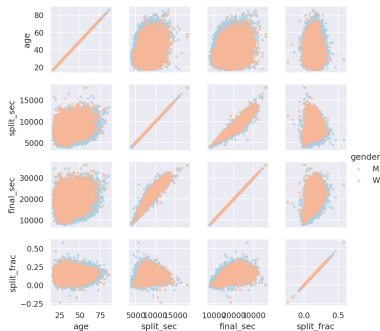
Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Khảo sát tương quan nhiều biến bằng `sns.PairGrid` có phân nhóm màu.
- 2 So sánh phân phối giữa các nhóm bằng `kdeplot` và `violinplot`.
- 3 Diễn giải kết quả hồi quy một cách thận trọng (tương quan $\neq$ nhân quả).

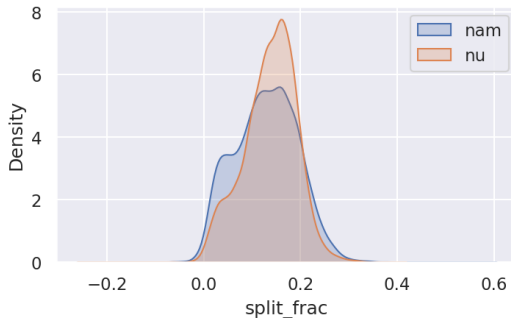
PairGrid: tương quan đa biến theo giới tính

```
g = sns.PairGrid(data, vars=['age','split_sec','final_sec','split_frac'],  
                 hue='gender', palette='RdBu_r')  
g.map(plt.scatter, alpha=0.6); g.add_legend()
```



Phân phối split_frac theo giới tính

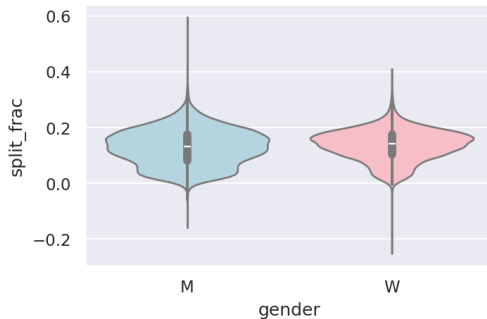
```
sns.kdeplot(data.split_frac[data.gender=='M'], label='nam', fill=True)  
sns.kdeplot(data.split_frac[data.gender=='W'], label='nu', fill=True)
```



Nam giới có xu hướng phân bố *gần đều tốc độ hơn* (đỉnh nhọn gần 0); phân phối 2 giới đều hơi lệch phải (đa số chậm lại).

Violin plot: so sánh hình dạng phân phối

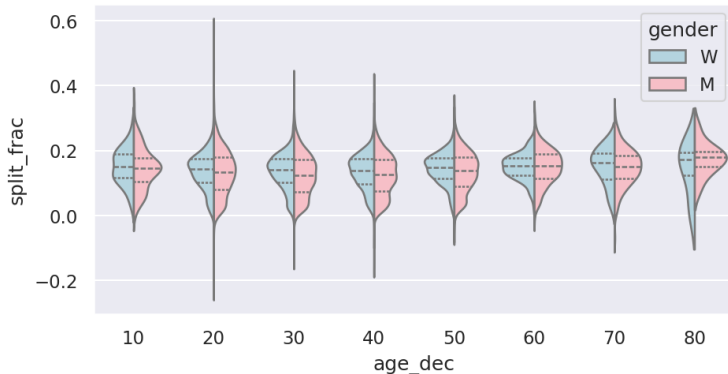
```
sns.violinplot(x='gender', y='split_frac', data=data,  
               palette=['lightblue', 'lightpink']);
```



Violin plot = boxplot (Module 44) + KDE (Module 41) đặt đối xứng – vừa thấy tứ phân vị, vừa thấy hình dạng phân phối.

Violin plot chia đôi theo độ tuổi

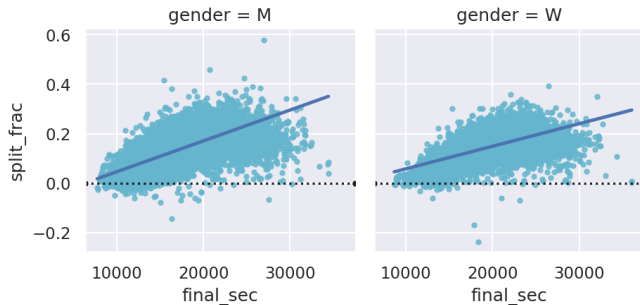
```
data['age_dec'] = data.age.map(lambda age: 10 * (age // 10))
sns.violinplot(x='age_dec', y='split_frac', hue='gender', data=data,
               split=True, inner='quartile');
```



`split=True`: tách đôi mỗi violin theo hue, so sánh 2 nhóm trên cùng 1 trục.

Hồi quy theo nhóm với lmpplot

```
g = sns.lmplot(x='final_sec', y='split_frac', col='gender', data=data,  
               markers='.', scatter_kws=dict(color='c'))  
g.map(plt.axhline, y=0.0, color='k', ls=':')
```



Diễn giải kết quả: tương quan không phải nhân quả

Tư duy phản biện

Người có `split_frac` gần 0 (tốc độ đều) có xu hướng về đích nhanh hơn (`final_sec` thấp). Điều này **không chứng minh** “giữ tốc độ đều” *gây ra* kết quả tốt – nhiều khả năng *cả hai* đều là hệ quả của một yếu tố ẩn: **thể lực/kinh nghiệm luyện tập** của vận động viên. Đây là ví dụ điển hình về *biến gây nhiễu* (confounding variable), sẽ gặp lại khi học hồi quy thống kê ở Buổi 13.

- `sns.PairGrid + hue=`: mở rộng `pairplot` với toàn quyền kiểm soát hàm vẽ từng ô.
- `sns.violinplot(..., split=True)`: so sánh 2 phân phối trên cùng 1 trục, tiết kiệm không gian.
- `sns.lmplot(..., col=)`: hồi quy riêng cho từng nhóm, xếp cạnh nhau để so sánh.
- Luôn diễn giải tương quan một cách thận trọng – kiểm tra khả năng có biến gây nhiễu.

Module 50: Trực quan hóa 3 chiều (3D) – Cơ bản

Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

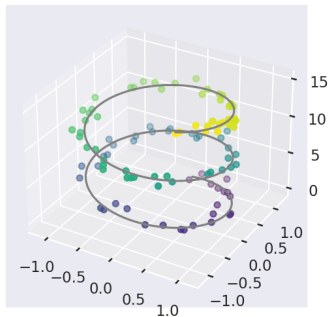
- 1 Tạo hệ trục 3D bằng `projection='3d'`.
- 2 Vẽ đường và điểm 3D bằng `plot3D/scatter3D`.
- 3 Vẽ contour 3D và điều chỉnh góc nhìn bằng `view_init()`.

Ghi chú

`mplot3d` là bộ công cụ 3D tích hợp sẵn của Matplotlib – đơn giản, đủ dùng cho khảo sát nhanh, nhưng không mạnh bằng các thư viện 3D chuyên dụng (Plotly, Mayavi).

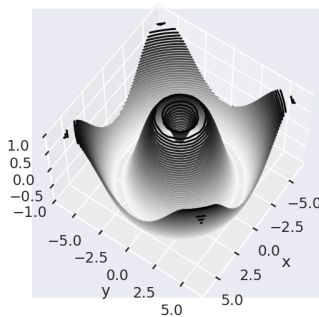
Điểm và đường trong không gian 3D

```
from mpl_toolkits import mplot3d  
ax = fig.add_subplot(projection='3d')  
ax.plot3D(xline, yline, zline, 'gray')  
ax.scatter3D(xdata, ydata, zdata, c=zdata, cmap='viridis');
```



Contour 3D và góc nhìn tùy chỉnh

```
X, Y = np.meshgrid(x, y)                                # nhu Buoi 8 (Module 33)
Z = np.sin(np.sqrt(X**2 + Y**2))
ax = fig.add_subplot(projection='3d')
ax.contour3D(X, Y, Z, 40, cmap='binary')
ax.view_init(60, 35)                                     # goc nhin: elevation, azimuth
```



Tổng kết Module 50

- `fig.add_subplot(projection='3d')`: cách hiện đại tạo hệ trục 3D (thay cho `plt.axes(projection='3d')` cũ).
- `plot3D/scatter3D`: tương đương `plot/scatter` 2D, thêm 1 tham số tọa độ z.
- `ax.contour3D()`: tái sử dụng `np.meshgrid` đã học ở Module 33 (Buổi 8), chỉ đổi hệ trục.
- `view_init(elev, azim)`: xoay góc nhìn – quan trọng vì hình 3D tĩnh dễ gây hiểu nhầm nếu chọn góc xấu.

Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

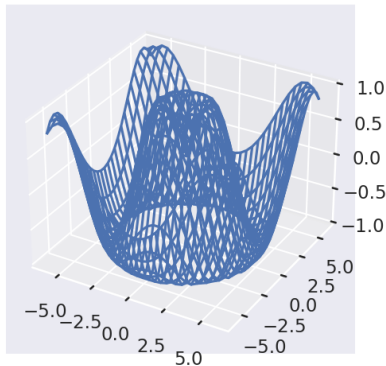
- 1 Phân biệt wireframe và surface plot.
- 2 Dùng `plot_trisurf()` khi dữ liệu không nằm trên lưới đều.

Wireframe và Surface Plot

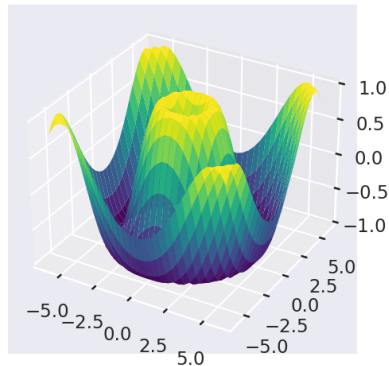
```
ax.plot_wireframe(X, Y, Z)  
ax.plot_surface(X, Y, Z, cmap='viridis', edgecolor='none')
```

```
# khung day  
# mat dac
```

wireframe



surface

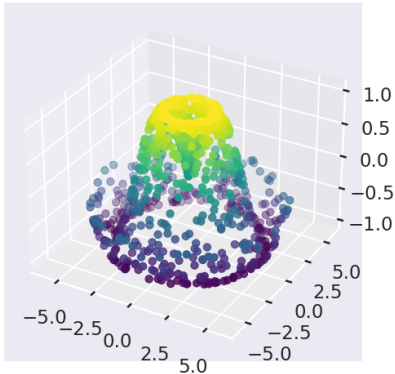


Surface tô đầy mỗi ô lưới bằng đa giác có màu – cảm nhận địa hình rõ hơn wireframe.

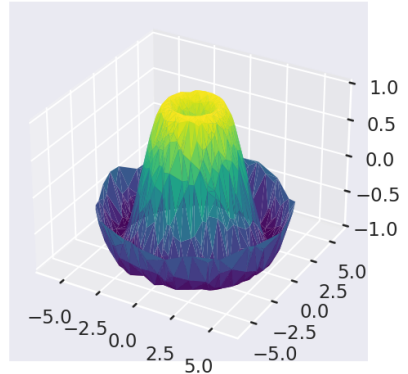
Khi dữ liệu không nằm trên lưới đều: plot_trisurf

```
theta = 2*np.pi*rng.random(1000); r = 6*rng.random(1000) # mau ngau nhien
ax.scatter(x, y, z, c=z, cmap='viridis') # (a) chỉ là mây điểm
ax.plot_trisurf(x, y, z, cmap='viridis', edgecolor='none') # (b) tam giác hoa
```

scatter (diem ngau nhien)

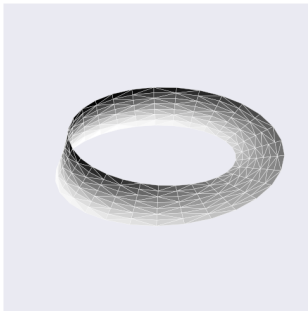


plot_trisurf



Ví dụ: dải Möbius (Möbius Strip)

```
tri = Triangulation(np.ravel(w), np.ravel(theta)) # tam giác hoa tren tham so hoa
ax.plot_trisurf(x, y, z, triangles=tri.triangles, cmap='Greys');
```



Dải giấy chỉ có *1 mặt* duy nhất – minh họa kinh điển cho sức mạnh của `plot_trisurf` khi kết hợp với tam giác hóa trên không gian tham số (không phải không gian (x, y, z) cuối cùng).

Tổng kết Module 51

- `plot_wireframe`: khung lưới, nhẹ, phù hợp xem nhanh cấu trúc.
- `plot_surface`: mặt đặc có màu, trực quan hơn nhưng nặng hơn khi lưới lớn.
- `plot_trisurf`: dùng tam giác hóa Delaunay – xử lý được dữ liệu *không đều*, không cần `meshgrid`.
- Tam giác hóa trên không gian *tham số* (trước khi ánh xạ sang (x, y, z)) là kỹ thuật mở ra nhiều hình 3D phức tạp (ví dụ dải Möbius).

Module 52: EDA case study lớn – US Baby Names (I)

Chuẩn đầu ra (Learning Outcomes)

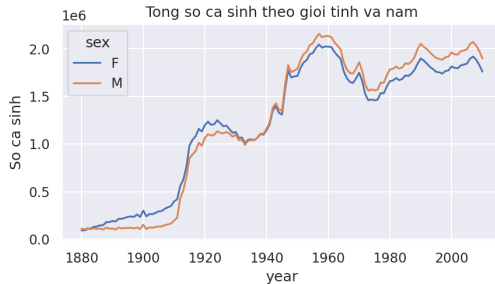
Sau module này, học viên có khả năng:

- ➊ Gộp (concat) hàng trăm file CSV thành 1 DataFrame lớn.
- ➋ Áp dụng lại toàn bộ chuỗi kỹ năng Buổi 3–9 trên 1 case study thật, quy mô lớn.

Dữ liệu thật: `datasets/babynames` (PDA, nguồn: Social Security Administration Hoa Kỳ) – 131 file (`yob1880.txt...yob2010.txt`), tổng **1 690 784** dòng.

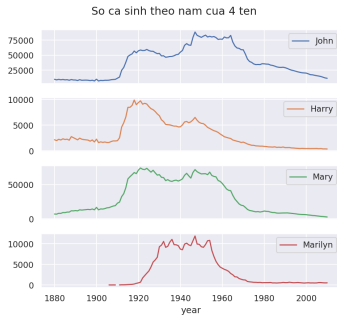
Gộp 131 file thành 1 DataFrame

```
pieces = []
for year in range(1880, 2011):
    frame = pd.read_csv(f"yob{year}.txt", names=["name","sex","births"])
    frame["year"] = year
    pieces.append(frame)
names = pd.concat(pieces, ignore_index=True)          # nhu Buoi 5 (Module 17)
total_births = names.pivot_table("births", index="year", columns="sex", aggfunc="sum")
```



Top 1000 tên phổ biến mỗi năm, mỗi giới tính

```
def get_top1000(group):  
    return group.sort_values("births", ascending=False)[:1000]  
  
top1000 = (names.groupby(["year","sex"])  
            .apply(get_top1000, include_groups=False)  
            .reset_index(level=["year","sex"]))
```



Lưu ý kỹ thuật: pandas 3.0 và `groupby.apply`

Thay đổi hành vi quan trọng

Kể từ pandas 3.0, `groupby(...).apply(ham)` **không còn truyền** các cột dùng để nhóm (ở đây là `year`, `sex`) vào bên trong `ham` nữa – tránh nhầm lẫn giữa dữ liệu và khóa nhóm. Cần: (1) thêm `include_groups=False` để tắt cảnh báo, và (2) gọi `.reset_index(level=[...])` sau đó để lấy lại các cột này từ `MultiIndex` kết quả.

Đây là ví dụ thực tế cho thấy **API thư viện thay đổi theo thời gian** – kỹ năng đọc changelog/deprecation warning quan trọng không kém việc thuộc cú pháp.

Tổng kết Module 52

- `pd.concat()` trên vòng lặp file: mẫu chuẩn gộp dữ liệu nhiều năm/nhiều nguồn (Buổi 5).
- `groupby().apply()`: trích top-N theo từng nhóm – tổng quát hơn `nlargest()`.
- Luôn kiểm tra phiên bản thư viện khi copy code từ sách/notebook cũ – hành vi API có thể đã thay đổi.

Module 53: EDA case study lớn – US Baby Names (II)

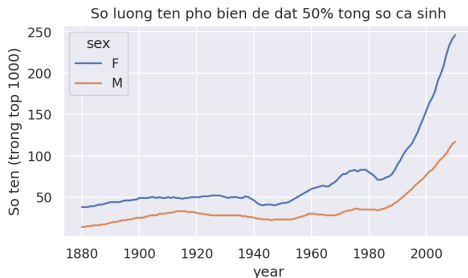
Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Định lượng “tính đa dạng” của một phân phối bằng ngưỡng tích lũy (cumulative sum).
- 2 Rút ra kết luận xã hội học từ một chuỗi biến đổi tuần tự trên dữ liệu (data storytelling).

Đo độ đa dạng: cần bao nhiêu tên để chiếm 50%?

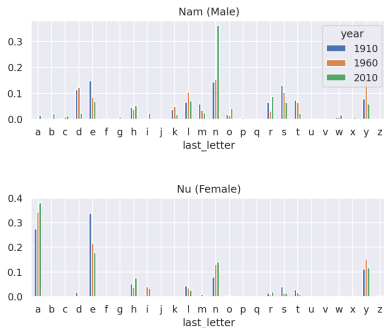
```
def get_quantile_count(group, q=0.5):  
    group = group.sort_values("prop", ascending=False)  
    return group.prop.cumsum().searchsorted(q) + 1  
  
diversity = top1000.groupby(["year", "sex"]).apply(get_quantile_count,  
                                                    include_groups=False)
```



Số tên cần thiết **tăng liên tục** qua 130 năm – xã hội ngày càng đặt tên đa dạng hơn, ít tên “áp đảo” như trước.

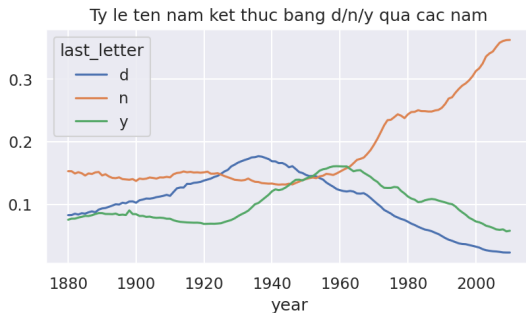
Xu hướng chữ cái cuối cùng của tên

```
last_letters = names["name"].map(lambda x: x[-1])  
table = names.pivot_table("births", index=last_letters,  
                           columns=["sex", "year"], aggfunc="sum")  
subtable = table.reindex(columns=[1910, 1960, 2010], level="year")
```



Theo dõi 3 chữ cái qua thời gian: d, n, y

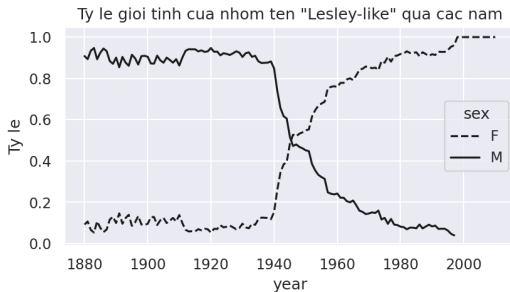
```
letter_prop = table / table.sum()  
dny_ts = letter_prop.loc[['d','n','y'], 'M'].T  
dny_ts.plot()
```



Tên nam kết thúc bằng "n" (ví dụ Aiden, Jayden, Mason) tăng mạnh từ 1960 – xu hướng đặt tên có thể quan sát trực tiếp qua 1 chữ cái.

Tên trung tính giới: trường hợp “Lesley”

```
all_names = pd.Series(top1000["name"].unique())
lesley_like = all_names[all_names.str.contains("Lesl")]
# ['Leslie', 'Lesley', 'Leslee', 'Lesli', 'Lesly']
filtered = top1000[top1000["name"].isin(lesley_like)]
table = filtered.pivot_table("births", index="year", columns="sex", aggfunc="sum")
```



Kết luận Module 53: từ dữ liệu đến câu chuyện

- Nhóm tên “Lesley” đảo chiều hoàn toàn từ chủ yếu **nam** (1880–1940) sang chủ yếu **nữ** (từ 1950) – hiện tượng “tên bị nữ hóa” (feminization) từng ghi nhận trong xã hội học đặt tên.
- Đây là ví dụ hoàn chỉnh về **EDA end-to-end**: nạp dữ liệu thô (131 file) → làm sạch/gộp → biến đổi (pivot, cumsum) → trực quan hóa → **diễn giải** thành một câu chuyện có ý nghĩa.
- Quy trình này – không phải riêng bộ dữ liệu Baby Names – chính là khuôn mẫu cho **Đồ án giữa kỳ**.

- `cumsum().searchsorted(q)`: kỹ thuật đo độ tập trung/đa dạng của một phân phối xếp hạng.
- `Series.str.contains()`: lọc theo mẫu chuỗi (Buổi 7) để nhóm các biến thể cùng 1 tên.
- Trực quan hóa tốt nhất luôn đi kèm **diễn giải bằng lời** – biểu đồ chỉ là phương tiện, câu chuyện rút ra mới là giá trị.

Bài tập 1: Mở rộng case study MovieLens

Dữ liệu: `datasets/movielens/` (`users.dat`, `ratings.dat`, `movies.dat` – 1 triệu đánh giá phim, đã dùng ở Buổi 6 cho `merge/groupby`).

- ➊ Gộp 3 bảng bằng `pd.merge()` (đã học Buổi 5), tính điểm trung bình mỗi phim theo giới tính người xem bằng `pivot_table`.
- ➋ Dùng `sns.histplot()` (Module 41) so sánh phân phối điểm đánh giá giữa nam và nữ.
- ➌ Dùng `sns.catplot(kind="bar")` (Module 44) vẽ 10 phim có *chênh lệch điểm theo giới tính* lớn nhất.
- ➍ Câu hỏi thảo luận: nếu 1 phim chỉ có 3 lượt đánh giá nhưng đạt 5 sao tuyệt đối, có nên xếp vào top phim hay/dở nhất không? Vì sao?

Bài tập 2: USDA Food Database & FEC 2012

- 1 **USDA Food** (`datasets/usda_food/database.json`): với mỗi nhóm chất dinh dưỡng (`nutgroup`), dùng `sns.kdeplot` (Module 41) so sánh phân phối giá trị dinh dưỡng giữa các nhóm thực phẩm (`fgroup`).
- 2 **FEC 2012** (`datasets/fec/P00000001-ALL.csv`): dùng `pd.cut()` (Buổi 6) chia mức đóng góp tranh cử thành các khoảng (bin), sau đó `sns.catplot(kind="bar")` so sánh tỷ trọng đóng góp giữa 2 ứng viên theo từng khoảng.
- 3 Cả hai đều là ví dụ cho thấy: kỹ năng `groupby/pivot_table` (Buổi 6) + Seaborn (Buổi 9) giải quyết trọn vẹn 1 bài toán EDA thực tế.

Bài tập 3: Câu hỏi thảo luận – chọn biểu đồ phù hợp

Với mỗi tình huống sau, chọn loại biểu đồ phù hợp nhất (Buổi 8 hoặc 9) và giải thích lý do:

- ➊ So sánh phân phối thu nhập giữa 5 tỉnh/thành khác nhau, cần thấy rõ cả trung vị lẫn hình dạng phân phối.
- ➋ Khảo sát tương quan giữa 6 biến số liên tục trong 1 bộ dữ liệu chưa quen thuộc.
- ➌ Trực quan hóa vị trí 10 000 giao dịch thương mại điện tử ghi nhận theo kinh độ/vĩ độ.
- ➍ Trình bày một bề mặt hàm chi phí (cost surface) 2 tham số trong bài toán tối ưu hóa.

Gợi ý đáp án: (1) violin plot – Module 49; (2) pairplot/PairGrid – Module 42, 49; (3) scatter địa lý – Module 47; (4) surface plot – Module 51.

Bảng tổng hợp các hàm Seaborn & 3D trong Buổi 9

Nhóm	Hàm/thao tác	Mục đích
Nền tảng	<code>sns.set_theme</code> , <code>sns.axes_style</code>	Đổi giao diện mặc định/tạm thời
Phân phối	<code>sns.histplot</code> , <code>kdeplot</code> , <code>displot</code>	Histogram, KDE 1D/2D
Đa biến	<code>sns.pairplot</code> , <code>PairGrid</code>	Ma trận tương quan nhiều biến
Lưới facet	<code>sns.FacetGrid</code>	Chia nhỏ theo biến phân loại
Phân loại	<code>sns.catplot</code> (box/count/violin)	So sánh theo nhóm rời rạc
Đồng thời	<code>sns.jointplot</code> (hex/reg)	Quan hệ 2 biến + phân phối biên
Hồi quy	<code>sns.lmplot</code>	Hồi quy tuyến tính tự động theo nhóm
3D	<code>plot3D</code> , <code>contour3D</code> , <code>plot_surface</code> , <code>plot_trisurf</code>	Trực quan hóa không gian 3 chiều

Yêu cầu tổng quát

Áp dụng quy trình EDA end-to-end (như Module 52–53) trên **1 bộ dữ liệu thật do sinh viên tự chọn**, hoặc 1 trong các case study đã giới thiệu (bit.ly/USA.gov, Marathon, Baby Names, MovieLens, USDA Food, FEC).

- ➊ Nạp và làm sạch dữ liệu (Buổi 3–7): xử lý thiếu dữ liệu, kiểu dữ liệu, gộp bảng nếu cần.
- ➋ Tổng hợp/biến đổi (Buổi 6): `groupby`, `pivot_table`, biến phái sinh.
- ➌ Trực quan hóa (Buổi 8–9): tối thiểu 4 loại biểu đồ khác nhau, có ít nhất 1 biểu đồ Seaborn đa biến.
- ➍ Diễn giải: mỗi biểu đồ đi kèm 2–3 câu nhận xét/kết luận bằng ngôn ngữ tự nhiên.

Cấu trúc nộp bài đề xuất

- ➊ **Giới thiệu dữ liệu:** nguồn gốc, số dòng/cột, ý nghĩa các biến chính.
- ➋ **Làm sạch & tiền xử lý:** các bước xử lý, lý do lựa chọn (ví dụ: loại outlier bằng IQR như Buổi 8).
- ➌ **Khám phá bằng biểu đồ:** theo trình tự từ tổng quan (pairplot) đến chi tiết (facet/categorical).
- ➍ **Kết luận:** 3–5 phát hiện chính, có thể đề xuất câu hỏi cho giai đoạn Machine Learning (Buổi 10–12).

Notebook Jupyter nộp kèm mã nguồn đầy đủ, có thể chạy lại từ đầu (Kernel → Restart & Run All) – tiêu chí đánh giá bắt buộc.

Tài liệu tham khảo I

- VanderPlas, J. (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly Media.
- McKinney, W. (2022). *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter* (3rd ed.). O'Reilly Media.
- Waskom, M. L. (2021). Seaborn: statistical data visualization. *Journal of Open Source Software*, 6(60), 3021.
- VanderPlas, J. (n.d.). *PythonDataScienceHandbook* [Kho mã nguồn – notebooks 04.12, 04.14; dữ liệu marathon]. GitHub. <https://github.com/jakevdp/PythonDataScienceHandbook>
- VanderPlas, J. (n.d.). *marathon-data* [Bộ dữ liệu]. GitHub. <https://github.com/jakevdp/marathon-data>
- McKinney, W., et al. *python-for-data-analysis* [Kho mã nguồn – ch13.ipynb, ch13-2.ipynb, ch13-3.ipynb; dữ liệu bit.ly/USA.gov, MovieLens, Baby Names]. GitHub.
- GroupLens Research. *MovieLens 1M Dataset*. University of Minnesota.
- U.S. Social Security Administration. *Baby Names from Social Security Card Applications, 1880–2010* [Bộ dữ liệu].
- The Matplotlib development team. *mplot3d Toolkit documentation*. <https://matplotlib.org>

Chuẩn bị cho buổi tiếp theo: Nhập Môn Machine Learning

Thông điệp

Buổi 8–9 đã trang bị trọn bộ kỹ năng trực quan hóa và EDA – nền tảng bắt buộc trước khi xây dựng mô hình. Buổi 10 chuyển sang Scikit-Learn: Estimator API (liên hệ trực tiếp OOP đã học ở Buổi 1 và hồi quy tuyến tính tự cài đặt ở Buổi 2), model validation, và feature engineering. Đề án giữa kỳ (đề xuất ở Buổi 9) sẽ được mở rộng thêm bước *mô hình hóa* khi học xong Buổi 10–12.